

MODULE 02

SYLLABUS: Concurrency and Synchronization - Basic principles, Mechanisms - Locks: The Basic Idea, Building Spin Locks with Test And -Set, Compare and Swap, Using Queues: Sleeping Instead Of Spinning, Semaphores - Definition, Binary Semaphores, The Producer/Consumer (Bounded Buffer) Problem and its solution using semaphores, Reader-Writer Locks. Case study: Linux Kernel Synchronization Methods - Spin Locks, Semaphores, Mutexes. Concurrency: Deadlock and Starvation - Deadlock Characterization, Deadlock Prevention and Avoidance, Deadlock Detection and recovery, Dining Philosophers Problem and its solution.

TO WATCH VIDEO CLASSES
OF THIS COURSE: [TAP HERE OR
SCAN QR CODE](#)



BASIC PRINCIPLES & LOCKS

Before diving into synchronization, two foundational ideas must be clear.

Concurrent Execution means multiple processes run simultaneously (or appear to), sharing the CPU through context switching. A process can be interrupted at any point mid-execution — even between two simple instructions like `register = counter` and `counter = register`.

Shared Resources are memory locations, variables, files, or buffers accessed by more than one process. When multiple processes read and write the same data concurrently without control, the results become unpredictable. This is exactly the problem that synchronization mechanisms solve.

Process Synchronization — Basic Principles

Independent vs. Cooperating Processes

Processes executing concurrently in the OS fall into two categories:

Independent Process: A process that cannot affect or be affected by other processes executing in the system. It has no shared data with anyone.

Cooperating Process: A process that can affect or be affected by other processes. They share resources such as memory, variables, code, and files.

Cooperating processes are powerful — they allow data sharing and parallel work — but they introduce a critical danger: if two processes access and modify the same data at the same time without coordination, the data can become corrupted.

The Producer-Consumer Problem

The Producer-Consumer problem is a classical synchronization problem where one process (producer) generates data and another process (consumer) uses it, sharing a common bounded buffer.

The producer places items into a shared buffer; the consumer removes items from it. To track how many items are in the buffer, an integer variable counter is maintained — incremented by the producer after each addition, decremented by the consumer after each removal.

Producer code:

```
while (true) {  
    while (counter == BUFFER_SIZE) ; // wait if full  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++; // signal item added  
}
```

Consumer code:

```
while (true) {  
    while (counter == 0) ; // wait if empty  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--; // signal item removed  
}
```

On the surface this looks correct, but it is **not**, because `counter++` and `counter--` are not truly single atomic operations at the machine level.

Race Condition

A race condition is a situation where several processes access and manipulate the same shared data concurrently, and the final outcome depends on the particular order in which the accesses take place.

To understand why this is dangerous, consider that `counter++` at the machine level is actually three steps:

```
register1 = counter  
register1 = register1 + 1  
counter = register1  
Similarly, counter-- is:  
register2 = counter  
register2 = register2 - 1  
counter = register2
```

Now suppose `counter = 5` and both producer and consumer execute concurrently. One possible interleaving:

Step	Who	Operation	Value
S0	Producer	register1 = counter	register1 = 5
S1	Producer	register1 = register1 + 1	register1 = 6
S2	Consumer	register2 = counter	register2 = 5
S3	Consumer	register2 = register2 - 1	register2 = 4
S4	Producer	counter = register1	counter = 6
S5	Consumer	counter = register2	counter = 4 ← Wrong!

The correct answer is 5, but the result is 4. Depending on the interleaving, the result can be 4, 5, or 6 — all from the same initial state. This unpredictability is the race condition. The solution is **process synchronization**.

The Critical Section Problem

A **critical section** is the portion of code in a process that accesses shared resources — such as shared variables, tables, or files — and whose concurrent execution by multiple processes may lead to a race condition.

The rule is simple: **only one process may execute inside its critical section at any given time.**

General Structure of a Process

Every process that uses a critical section follows this structure:

```
do {  
    entry section    ← request permission to enter  
    critical section ← shared resource access  
    exit section     ← signal that CS is done  
    remainder section ← rest of the code  
} while (true);
```

Three Requirements for a Valid Solution

Any correct solution to the critical section problem must satisfy all three of the following conditions:

- 1. Mutual Exclusion:** If process P_i is executing in its critical section, no other process may simultaneously execute in its critical section. This is the most fundamental requirement.
- 2. Progress:** If no process is currently in its critical section and some processes wish to enter, the decision of who enters next cannot be postponed indefinitely. A process not interested in entering the CS must not block others from entering.
- 3. Bounded Waiting:** After a process has made a request to enter its critical section, there must be a limit on how many times other processes are allowed to enter before that request is granted. This prevents **starvation**.

Software vs. Hardware Solutions

Feature	Software Solutions	Hardware Solutions
Implementation	Entirely in code using shared variables	Uses special atomic CPU instructions
Example	Peterson's Algorithm	Test-and-Set, Compare-and-Swap
Atomicity	Not inherently guaranteed	Guaranteed by hardware
Complexity	More complex, error-prone	Simpler, but lower-level
Bounded Waiting	Can be designed to satisfy	Not always guaranteed

Peterson's Solution (Software Solution)

Peterson's Solution is a classic software-based algorithm that solves the critical section problem for two processes using two shared variables.

Variables Used:

- `int turn` — indicates whose turn it is to enter the critical section.
- `boolean flag[2]` — `flag[i] = true` means process P_i is ready and wishes to enter its critical section.

Algorithm:

Process P_i :

```
do {
    flag[i] = true;           // I am ready
    turn = j;                 // But I give priority to j
    while (flag[j] && turn == j); // Wait if j wants to enter
    critical section
    flag[i] = false;          // I am done
    remainder section
} while (true);
```

Process P_j :

```
do {
    flag[j] = true;
    turn = i;
    while (flag[i] && turn == i);
    critical section
    flag[j] = false;
    remainder section
} while (true);
```

Why It Works — CS Requirements Check:

1. **Mutual Exclusion:** P_i enters the CS only when either $\text{flag}[j] == \text{false}$ (P_j is not interested) or $\text{turn} == i$ (it is P_i 's turn). Both cannot be true simultaneously for both processes.
2. **Progress:** If P_j is not interested ($\text{flag}[j] = \text{false}$), P_i enters immediately without waiting.
3. **Bounded Waiting:** Since turn can only be either i or j , once a process yields the turn to the other, the other will enter within one cycle. No process waits forever.

Important Note: Peterson's Solution may not work correctly on modern processors due to instruction reordering by the CPU or compiler. However, it remains an excellent conceptual model that clearly illustrates mutual exclusion, progress, and bounded waiting.

Locks — The Basic Idea

What is a Lock?

A lock is a synchronization variable that can be in one of two states — **available (free/unlocked)** or **acquired (held/locked)** — used to protect a critical section by ensuring only one thread enters at a time.

The programmer wraps the critical section with two operations:

- `lock()` — acquire the lock before entering.
- `unlock()` — release the lock after exiting.

```
do {
    acquire lock
    critical section
    release lock
    remainder section
} while (TRUE);
```

When a thread calls `lock()` and the lock is free, it acquires it and enters the critical section — this thread is called the **owner** of the lock. If the lock is already held, the calling thread must wait. When the owner calls `unlock()`, the lock becomes free and one waiting thread may proceed.

Locks give the programmer direct control over scheduling: no more than one thread can ever be active within the locked code region.

Simple Lock (Lock Variable Approach)

The simplest lock uses an integer variable, initialized to 0.

- Lock = 0 → critical section is **free**.
- Lock = 1 → critical section is **occupied**.

Entry Section: while (Lock != 0) ; Lock = 1;

Critical Section

Exit Section: Lock = 0;

Working (Scene by scene):

Scene 1 (P₀ arrives, Lock = 0): P₀ checks the condition — Lock is 0, so the while loop exits. P₀ sets Lock = 1 and enters the CS. Even if P₀ is preempted mid-CS, no other process can enter since Lock remains 1.

Scene 2 (P₁ arrives while P₀ is in CS): P₁ checks Lock — it is 1, so the while loop condition is true. P₁ is trapped in an infinite spin, repeatedly checking until Lock becomes 0.

Scene 3 (P₀ exits CS): P₀ sets Lock = 0. P₁'s while loop condition breaks, P₁ sets Lock = 1 and enters the CS.

Failure of this Mechanism: Despite appearing correct, this simple lock **cannot guarantee mutual exclusion**. If both processes check Lock simultaneously (both see 0), both exit the while loop and both set Lock = 1 — two processes are now in the CS simultaneously. This is a classic race condition. The check and set must be done as a **single atomic operation**, which software alone cannot guarantee. This motivates **hardware solutions**.

Model Question

Processes P1 and P2 use the following code for synchronization. Assume flag is a shared variable initialized to false. (3 Marks)

```
if (flag == false)
{
    flag = true;
    criticalSection();
    flag = false;
}
```

Does this guarantee mutual exclusion? Can it lead to deadlock?

Solution

No, this solution does not guarantee mutual exclusion.

Reason:

The statements: if(flag == false) and flag = true are not executed atomically.

Suppose both P1 and P2 check flag simultaneously:

- Both see flag = false
- Both enter the if block
- Both set flag = true
- Both enter critical section together

Thus, mutual exclusion is violated.

Deadlock: No deadlock occurs because processes do not wait indefinitely for each other. They either enter the critical section or skip the if condition.

Building Spin Locks — Hardware Solutions

The Need for Atomic Instructions

To correctly implement synchronization, the **test** (read the old value) and **set** (write the new value) must happen as one uninterruptable atomic operation. The CPU provides special instructions for this purpose.

Test-and-Set (TAS) Instruction

`test_and_set` is an atomic hardware instruction that reads the current value of a memory location, sets it to **TRUE**, and returns the original value — all in a single uninterruptable operation.

```
boolean test_and_set(boolean *target) {
    boolean rv = *target; // save old value
    *target = TRUE;       // set to TRUE unconditionally
    return rv;            // return old value
}
```

Three key properties:

1. Executed **atomically** — cannot be interrupted midway.
2. Returns the **original** value of the target.
3. Always sets target to **TRUE**.

Solution using test_and_set (Spin Lock):

Shared variable: `boolean lock = false` (initially free)

```
do {
    while (test_and_set(&lock)) ; // spin if lock was TRUE (busy)
    /* critical section */
    lock = false;                // release lock
    /* remainder section */
} while (true);
```

How it works: When `lock = false`, `test_and_set` returns false (loop exits) and sets `lock = true`. The process enters the CS. Any other process calling `test_and_set` now gets true returned (lock is held), so it keeps spinning. When the CS process sets `lock = false`, a waiting process can finally enter.

CS Requirements Check:

Requirement	Satisfied?	Reason
Mutual Exclusion	✓ Yes	Only one process sees false returned; lock is immediately set to true atomically
Progress	✓ Yes	Once lock is set to false, another process enters immediately
Bounded Waiting	✗ No	No queue — any process that checks at the right moment can grab the lock; starvation possible

Compare-and-Swap (CAS) Instruction

`compare_and_swap` is an atomic instruction that checks whether a memory location holds an expected value, and only if it does, replaces it with a new value. It always returns the original value.

```
int compare_and_swap(int *value, int expected, int new_value) {
    int temp = *value;
    if (*value == expected)
        *value = new_value; // swap only if condition met
    return temp;
}
```

Three key properties:

1. Executed **atomically**.
2. Returns the **original** value.
3. Updates the value **only if** it equals the expected value.

Solution using `compare_and_swap`:

Shared variable: `int lock = 0` (0 = free, 1 = held)

```
do {
    while (compare_and_swap(&lock, 0, 1) != 0) ; // spin if lock was 1
    /* critical section */
    lock = 0; // release lock
    /* remainder section */
} while (true);
```

How it works: CAS is called with `expected = 0`, `new_value = 1`. If `lock` is 0 (free), the swap happens, `lock` becomes 1, and CAS returns 0 — the while condition fails and the process enters. If `lock` is already 1, the condition `1 == 0` is false, no swap occurs, and CAS returns 1 — the process keeps spinning.

CS Requirements Check:

Requirement	Satisfied?	Reason
Mutual Exclusion	✓ Yes	CAS atomically sets <code>lock</code> to 1; only one process sees the 0→1 transition
Progress	✓ Yes	When <code>lock</code> is reset to 0, another waiting process can enter
Bounded Waiting	✗ No	No ordering — any process that spins may keep winning the CAS race

Spin Locks — Evaluation

A spinlock is a low-level synchronization primitive where a thread busy-waits in a loop, continuously checking if the lock is free, rather than sleeping.

Both TAS and CAS implement spin locks. The process literally "spins" — running in a tight loop — until the lock becomes available.

Advantages:

- No context switch required while waiting → good for very short critical sections where the wait time is less than the cost of a context switch.
- Highly efficient on multi-core systems for brief waits.

Disadvantages:

- Wastes CPU cycles — a spinning thread consumes the CPU while doing nothing productive.
- On a single-CPU system, spinning is counterproductive: the thread holding the lock cannot even run to release it.
- No fairness guarantee — a thread may spin forever (starvation).

Evaluation Summary:

Criterion	Spin Lock
Correctness (Mutual Exclusion)	✓ Yes
Fairness (Bounded Waiting)	✗ No
Performance (Single CPU)	✗ Poor
Performance (Multi-core, short CS)	✓ Good

Using Queues: Sleeping Instead of Spinning

Problem with Pure Spin Locks

Spin locks have two critical weaknesses: they **waste CPU cycles** (busy waiting), and they offer **no fairness guarantee** — a spinning thread may never acquire the lock if other threads keep winning. The root cause is that spin locks rely entirely on the scheduler to decide which thread runs, with no control over who gets the lock next.

The Improved Approach — Queue + OS Support

Instead of spinning uselessly, a thread that cannot acquire the lock should **sleep** (be taken off the CPU), and be **explicitly woken** when the lock becomes free. To ensure fairness, waiting threads are maintained in a queue.

Two OS primitives are used:

- `park()` — puts the calling thread to sleep.
- `unpark(threadID)` — wakes a specific sleeping thread.

The lock structure uses three components:

- `flag` — indicates if the lock is held (1) or free (0).
- `guard` — a small spin lock protecting the flag and queue (spun on for only a few instructions, not the entire CS).
- `queue` — stores IDs of waiting threads in order.

Lock and Unlock Operations

Lock (`lock()`) logic:

1. Spin briefly on guard to get exclusive access to lock internals.
2. If `flag == 0` (lock is free): set `flag = 1`, release guard → enter CS.
3. If `flag == 1` (lock is busy): add thread ID to queue, release guard, call `park()` → thread sleeps.

Unlock (`unlock()`) logic:

1. Spin briefly on guard.
2. If queue is **empty**: set `flag = 0` (lock is now free).
3. If queue has **waiting threads**: call `unpark(next_thread_id)` — the lock is passed directly to the woken thread without resetting `flag` to 0 first (this prevents a race condition).

Code:

```
typedef struct __lock_t {
    int flag; // 0 = free, 1 = held
    int guard; // small spinlock for protecting internals
    queue_t *q; // waiting thread queue
} lock_t;
```

```

void lock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1) ; // acquire guard
    if (m->flag == 0) {
        m->flag = 1; // acquire lock
        m->guard = 0;
    } else {
        queue_add(m->q, getpid()); // add self to wait queue
        m->guard = 0;
        park(); // go to sleep
    }
}

void unlock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1) ; // acquire guard
    if (queue_empty(m->q))
        m->flag = 0; // no one waiting, free the lock
    else
        unpark(queue_remove(m->q)); // wake next thread, pass lock
    m->guard = 0;
}

```

Key Design Points

Why guard must be released before park(): If the guard is not released before calling park(), the thread sleeps while holding the guard. No other thread can then call unlock() (it would spin on guard forever), so the sleeping thread would never be woken — a **deadlock**.

Why flag is not reset to 0 when waking a thread: The lock is passed directly from the releasing thread to the next thread in the queue. Resetting flag to 0 would open a window where a third thread could also see flag = 0 and grab the lock, breaking mutual exclusion.

The Wakeup/Waiting Race Problem

A subtle race exists: a thread may decide to call park() (it has checked the flag and found it busy), but before it actually sleeps, the lock holder calls unpark(). The wakeup signal is missed, and the thread sleeps forever.

Solution — setpark(): Before releasing the guard, the thread calls setpark(), which marks it as "about to sleep." If unpark() arrives before park() actually executes, park() returns immediately instead of sleeping. This eliminates the missed-wakeup problem.

Comparison: Spin Lock vs. Queue-Based Lock

Feature	Spin Lock	Queue-Based (Sleep) Lock
Waiting mechanism	Busy-wait (CPU spinning)	Sleep (park/unpark)
CPU waste	High	None while sleeping
Fairness	No guarantee	FIFO queue → fair
Context switch	Not needed	Required for sleep/wake
Best for	Very short CS, multi-core	Long CS or single-CPU
Starvation	Possible	Eliminated by queue order

This queue-based lock is essentially a **hybrid lock**: it uses a tiny spin (on the guard, for just a few instructions) to safely access internal state, combined with sleep/wakeup primitives for actual waiting. The result is an efficient, fair, and robust lock — minimal spinning + explicit scheduling control via queue and OS primitives.

Model Question

A multithreaded web server maintains a cache of recently requested pages using a hash table with 1024 buckets. A hash function maps each URL to a bucket. Each bucket points either to NULL or to a doubly linked list of cached pages that must be searched sequentially. The cache supports three operations: insert, lookup, and delete. It is proposed to use mutex locks for synchronization. Briefly discuss the pros and cons for each of the following locking strategies. (6 Marks)

- (i) One mutex for the entire cache table
- (ii) One mutex per bucket
- (iii) One mutex per bucket and one per node in the linked list.

Solution

(i) One mutex for entire cache table

Advantages:

- Very simple to implement
- Easy to maintain correctness

Disadvantages:

- Only one thread can access the table at a time
- Poor concurrency
- High contention
- Performance decreases with many threads

(ii) One mutex per bucket

Advantages:

- Threads accessing different buckets can work simultaneously
- Better concurrency
- Reduced contention

Disadvantages:

- More memory required for multiple mutexes
- Slightly more complex implementation

(iii) One mutex per bucket and one per node

Advantages:

- Highest concurrency
- Threads can operate on different nodes simultaneously
- Better scalability

Disadvantages:

- Very complex implementation
- Higher locking overhead
- Increased risk of deadlock if locks are acquired improperly

Semaphores

A **semaphore** is a software-based process synchronization mechanism — an object with an integer value that can be manipulated using two atomic operations: `wait()` (also called `down()` or `P()`) and `signal()` (also called `up()` or `V()`). Proposed by Edsger Dijkstra in 1968, it provides more sophisticated synchronization than mutex locks.

The key insight of a semaphore is its **integer value**: this value determines whether a process may proceed or must wait. Before any operation, the semaphore must be initialized to some value, because that initial value defines its behavior.

```
#include <semaphore.h>
sem_t s;
sem_init(&s, 0, 1); // initialize semaphore s to 1
                  // 0 = shared between threads of same process
```

The Two Operations

```
int sem_wait(sem_t *s) {
    decrement the value of semaphore s by one
    wait if value of semaphore s is negative
}

int sem_post(sem_t *s) {
    increment the value of semaphore s by one
    if there are one or more threads waiting, wake one
}
```

The logic is precise and important:

- `sem_wait()` decrements the value first. If the result is negative, the calling thread is put to sleep and placed in a waiting queue. It does not consume CPU while waiting — this is the key advantage over spin locks.
- `sem_post()` increments the value. If there are sleeping threads, it wakes one of them.
- **General rule:** if the semaphore value is negative, its absolute value equals the number of currently waiting threads. For example, a value of -2 means 2 threads are sleeping and waiting.

Types of Semaphores

1. Binary Semaphore

A binary semaphore is one whose value can only be 0 or 1 (logically), initialized to 1. It functions exactly like a mutex lock — enforcing mutual exclusion so that only one process enters the critical section at a time.

- $S = 1 \rightarrow$ lock is **free**, critical section is available.
- $S = 0 \rightarrow$ lock is **held**, a process is in the critical section.

Usage pattern:

```
sem_wait(S)    ← acquire (entry section)
critical section
sem_post(S)    ← release (exit section)
```

2. Counting Semaphore

A counting semaphore has an integer value that can range over an unrestricted domain (initialized to N , where N is the number of available resources). It is used to control access to a resource that has multiple instances.

For example, if 3 printers are available, initialize the semaphore to 3. Each process that wants to use a printer calls `sem_wait()`, reducing the count. When done, it calls `sem_post()`. When count reaches 0, all resources are busy and any further requesting process sleeps.

Feature	Binary Semaphore	Counting Semaphore
Value range	0 and 1	0 to N (unrestricted)
Purpose	Mutual exclusion (like mutex)	Resource pool management
Initial value	1	N (number of resource instances)

Semaphore Working — Step-by-Step Trace

Case 1: Single Thread (Table Trace)

Semaphore Value	Thread 0 Action
1	calls <code>sem_wait()</code>
0	<code>sem_wait()</code> returns — $\text{value} \geq 0$, so no wait
0	enters critical section
0	calls <code>sem_post()</code>
1	<code>sem_post()</code> returns — no one waiting, lock now free

With a single thread, the semaphore trivially ensures mutual exclusion.

Case 2: Two Threads Competing (Key Steps)

Step 1: Thread 0 calls `sem_wait()`. Value goes $1 \rightarrow 0$. Since $\text{value} \geq 0$, Thread 0 enters the critical section. Thread 1 is in Ready state but not yet running.

Step 2: Context switch occurs. Thread 1 calls `sem_wait()`. Value goes $0 \rightarrow -1$. Since $\text{value} < 0$, Thread 1 goes to **Sleeping** state and relinquishes the CPU.

Key insight: Semaphore = -1 means: lock is held AND 1 thread is waiting.

Step 3: Thread 0 resumes, finishes critical section, calls `sem_post()`. Value goes $-1 \rightarrow 0$. Since a thread is waiting, Thread 1 is woken up (Sleeping $\rightarrow$ Ready).

Step 4: Thread 1 is scheduled, `sem_wait()` returns, Thread 1 enters critical section.

Step 5: Thread 1 calls `sem_post()`. Value goes $0 \rightarrow 1$. No threads waiting. Lock is free.

Why Semaphores Are Effective as Locks?

- Mutual exclusion is guaranteed
- No busy waiting — threads sleep instead of spinning, freeing the CPU
- Works correctly on both single-processor and multi-processor systems
- The scheduler efficiently manages blocked (sleeping) threads

Model Question

A shared variable y , initially 10, is accessed by two concurrent processes A and B. Process A reads y from memory, adds 5 to it, and writes the result back to memory. Process B reads y from memory, subtracts 3 from it, and writes the result back to memory. Before reading y , each process performs a P (wait) operation on a counting semaphore T. After writing the updated value back to memory, the process performs a V (signal) operation on T. If the semaphore T is initialized to 1, what is the maximum possible value of y after both processes complete execution? Justify your answer. (3 Marks)

Solution

Since semaphore T is initialized to 1, only one process can access y at a time.

Initial value: $y = 10$

Possible execution:

Process A: $y = 10 + 5 = 15$

Process B: $y = 15 - 3 = 12$

Maximum possible value after both complete = 12

Because semaphore ensures mutual exclusion, race condition does not occur.

The Producer-Consumer Problem — Solution Using Semaphores

The bounded buffer (producer-consumer) problem requires that a producer and a consumer share a buffer of fixed size N. The producer must not add items to a full buffer; the consumer must not remove items from an empty buffer; and they must not access the buffer simultaneously.

Solution Design

Three semaphores are used:

- **mutex** — initialized to 1. Binary semaphore ensuring mutual exclusion when accessing the buffer (only one process in the buffer at a time).
- **empty** — initialized to N (buffer size). Counts empty slots. Producer waits if buffer is full.
- **full** — initialized to 0. Counts filled slots. Consumer waits if buffer is empty.

Producer:

```
do {  
    // produce an item  
    sem_wait(&empty);    // wait for an empty slot  
    sem_wait(&mutex);    // acquire buffer lock  
  
    // add item to buffer  
  
    sem_post(&mutex);    // release buffer lock  
    sem_post(&full);     // signal that one more item is available  
} while (true);
```

Consumer:

```
do {  
    sem_wait(&full);     // wait for a filled slot  
    sem_wait(&mutex);    // acquire buffer lock  
  
    // remove item from buffer  
  
    sem_post(&mutex);    // release buffer lock  
    sem_post(&empty);    // signal that one more slot is empty  
  
    // consume item  
} while (true);
```

Why This Works

The empty and full semaphores handle the **ordering** constraint — producer cannot proceed when buffer is full, consumer cannot proceed when buffer is empty. The mutex semaphore handles the **mutual exclusion** constraint — only one process accesses the buffer at a time. The combination of all three makes the solution correct and complete.

Important ordering rule: Always call `sem_wait(&mutex)` after `sem_wait(&empty/full)`, never before. Reversing this order causes a deadlock — a process holds the mutex while waiting on empty/full, and the other process cannot enter to signal it.

Reader-Writer Locks (Readers-Writers Problem)

A database is shared among several concurrent processes. Some processes only **read** the data (readers), while others **read and write** (writers). Two readers can safely access the database simultaneously — there is no conflict. However, if a writer and any other process (reader or writer) access the database at the same time, data corruption may occur. Therefore, writers need **exclusive access**, while readers can share access.

Solution Using Semaphores

Three shared variables are used:

- **mutex** — semaphore, initialized to 1. Protects mutual exclusion when updating readcount. Used only by readers.
- **wrt** — semaphore, initialized to 1. Used by both readers and writers. A writer acquires this; the first reader acquires this to block writers while readers are present.
- **readcount** — integer, initialized to 0. Tracks how many readers are currently reading.

Writer Process:

```
do {
    wait(wrt);          // exclusive access to data
    /* write the data */
    signal(wrt);        // release access
} while (true);
```

Reader Process:

```
do {
    wait(mutex);        // protect readcount update
    readcount++;
    if (readcount == 1) // first reader blocks writers
        wait(wrt);
    signal(mutex);

    /* read the data */

    wait(mutex);        // protect readcount update
    readcount--;
    if (readcount == 0) // last reader releases writers
        signal(wrt);
    signal(mutex);
} while (true);
```

How It Works

When the first reader arrives, it calls `wait(wrt)`, blocking any writer. Subsequent readers simply increment `readcount` without touching `wrt` — so multiple readers proceed simultaneously. When the last reader exits, it calls `signal(wrt)`, allowing a waiting writer to proceed. A writer always calls `wait(wrt)`, so it is blocked as long as any reader holds `wrt`.

Variations and Starvation

Variation	Description	Risk
First variation	No reader kept waiting unless a writer has already been granted access	Writers may starve
Second variation	Once a writer is ready, it performs the write as soon as possible	Readers may starve

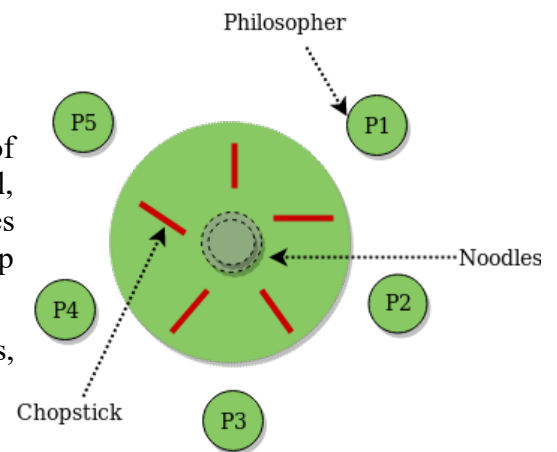
Both variations can lead to starvation, which is why some OS kernels directly provide **reader-writer locks** as a primitive to handle this more cleanly.

The Dining Philosophers Problem

Problem Description

Five philosophers sit around a circular table. Each has a bowl of food in front and one chopstick on each side (5 chopsticks total, shared between adjacent philosophers). A philosopher alternates between **thinking** and **eating**. To eat, a philosopher must pick up both the left and right chopstick. After eating, both are put down.

The mapping to OS concepts is: Philosophers = processes, Chopsticks = shared resources, Eating = using the resource.



Semaphore Solution

Each chopstick is represented by a binary semaphore, initialized to 1 (free).

```
semaphore chopstick[5]; // all initialized to 1

// Structure for Philosopher i:
do {
    wait(chopstick[i]);           // pick up left chopstick
    wait(chopstick[(i+1) % 5]);  // pick up right chopstick

    /* eat */

    signal(chopstick[i]);        // put down left chopstick
    signal(chopstick[(i+1) % 5]); // put down right chopstick

    /* think */
} while (TRUE);
```

The Deadlock Problem

This solution has a critical flaw. If all five philosophers become hungry simultaneously and each picks up their left chopstick, all five chopstick semaphores become 0. Now each philosopher waits for their right chopstick — but that is held by the neighbor who is also waiting. **All five philosophers wait forever — this is a deadlock.**

Possible Remedies

Three standard solutions exist to break the deadlock:

1. **Limit occupancy:** Allow at most **four philosophers** to sit at the table simultaneously. With only 4 philosophers competing for 5 chopsticks, at least one philosopher can always eat, guaranteeing progress.
2. **Both chopsticks or none:** A philosopher picks up chopsticks only if **both are available**. This check must be done inside a critical section (using a mutex) to prevent the scenario where two philosophers each grab one chopstick of a pair simultaneously.
3. **Asymmetric solution:** Break the symmetry in picking order.
 - **Odd-numbered** philosopher picks **left chopstick first**, then right.
 - **Even-numbered** philosopher picks **right chopstick first**, then left. This prevents the circular-wait condition that causes deadlock.

Model Question

Comment on the following solution to the dining philosophers problem. (3 Marks)

A hungry philosopher first picks up his left fork; if his right fork is also available, he picks up his right fork and starts eating; otherwise he puts down his left fork again and repeats the cycle.

Solution

In this solution, a philosopher picks up the left fork first and then the right fork if available.

Problem:

If all philosophers simultaneously pick up their left fork, each philosopher waits forever for the right fork held by the neighbor.

Thus:

- Circular wait occurs
- No philosopher can proceed
- Deadlock occurs

Hence, this solution is incorrect because it can lead to deadlock.

Semaphore Implementation — Busy Waiting Problem

The naive implementation of wait(S) uses a busy-wait loop:

```
wait(S) {
    while (S <= 0); // busy wait — wastes CPU
    S--;
}
signal(S) {
    S++;
}
```

This is the same spin lock problem discussed earlier. A semaphore that uses busy-waiting is called a **spinlock semaphore**. The correct implementation avoids this by using a **waiting queue** (as described in the queue-based sleeping lock section) — a blocked process is placed in a queue and put to sleep, removed from the CPU until it is explicitly woken by a signal().

Problems with Incorrect Semaphore Usage

Semaphores are powerful but dangerous when misused. Common errors:

- `signal(mutex)` before `wait(mutex)` — violates mutual exclusion; two processes may be in the CS simultaneously.
- `wait(mutex)` followed by another `wait(mutex)` without a signal — causes deadlock (the process holds the mutex and tries to acquire it again).
- Omitting `wait(mutex)` or `signal(mutex)` entirely — no protection, or permanent blocking.

All of these can lead to **deadlock** or **starvation**.

Case Study: Linux Kernel Synchronization Methods

Linux kernel uses synchronization methods to prevent race conditions and ensure safe access to shared data. Main synchronization methods:

1. Spin Locks
2. Semaphores
3. Mutexes

1. Spin Locks

A spin lock allows only one thread to enter the critical section at a time. If the lock is unavailable:

- The process continuously checks the lock
- This is called busy waiting or spinning

Features of Spin Locks

- Used for short critical sections
- Very fast
- No sleeping
- Used in interrupt handlers
- Not recursive

Basic Operations

Method	Purpose
<code>spin_lock()</code>	Acquire lock
<code>spin_unlock()</code>	Release lock
<code>spin_trylock()</code>	Try to acquire lock
<code>spin_is_locked()</code>	Check whether locked

Reader-Writer Spin Lock

Linux provides:

- `read_lock()`
- `write_lock()`

Rules:

- Multiple readers allowed
- Only one writer allowed
- Readers and writers cannot access simultaneously

2. Semaphores

A semaphore is a sleeping lock. If the semaphore is unavailable:

- Process sleeps
- CPU is given to another process

Features of Semaphores

- Suitable for long waiting time
- Process can sleep
- Cannot be used in interrupt context
- Uses wait queue

Semaphore Operations

Method	Purpose
sema_init()	Initialize semaphore
down_interruptible()	Acquire semaphore
down_trylock()	Try to acquire
up()	Release semaphore

Binary Semaphore

- Semaphore with value = 1
- Allows only one process at a time

Example: DECLARE_MUTEX(name);

3. Mutexes

A mutex is a mutual exclusion lock used to protect shared resources. Only one process can hold the mutex at a time.

Features of Mutex

- Sleeping lock
- More efficient than semaphore
- Only owner can unlock
- Recursive locking not allowed
- Cannot be used in interrupt context

Mutex Operations

Method	Purpose
mutex_lock()	Lock mutex
mutex_unlock()	Unlock mutex
mutex_trylock()	Try to lock
mutex_is_locked()	Check lock state

Difference Between Spin Lock, Semaphore and Mutex

Feature	Spin Lock	Semaphore	Mutex
Waiting Type	Busy waiting	Sleeping	Sleeping
Suitable For	Short wait	Long wait	Long wait
Interrupt Context	Allowed	Not allowed	Not allowed
Sleep While Holding	Not allowed	Allowed	Allowed
Speed	Very fast	Slower	Efficient

Choosing the Correct Synchronization Method

Requirement	Best Choice
Short critical section	Spin Lock
Long waiting time	Mutex
Interrupt handling	Spin Lock
Need sleeping	Mutex

Conclusion

Linux kernel synchronization methods prevent race conditions and ensure safe access to shared resources. Spin locks are suitable for short critical sections and interrupt handling, while semaphores and mutexes are used for longer waiting operations. Mutexes are generally preferred because they are simpler and more efficient for mutual exclusion.

DEADLOCKS

Before studying deadlocks, understand how processes use resources. A **resource** is anything a process needs to execute — CPU cycles, memory, I/O devices (printers, tape drives), files, etc. Each resource type may have one or more identical **instances** (e.g., two printers = two instances of resource type "printer").

Every process must follow this three-step resource-use cycle:

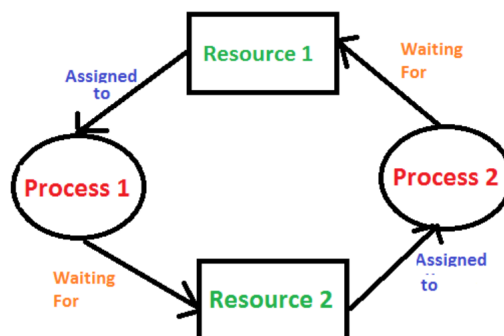
1. **Request** — Process asks the OS. If unavailable, it enters a wait state.
2. **Use** — Process operates on the resource (e.g., prints).
3. **Release** — Process frees the resource when done.

A problem arises when waiting processes each hold a resource needed by another — creating a circular dependency. This is called a **deadlock**.

What is Deadlock?

A deadlock is a situation in which a set of processes are permanently blocked because each process is holding at least one resource and waiting to acquire a resource currently held by another waiting process in the same set. Since no process can proceed or release its resources, the entire set remains indefinitely suspended.

Simple example: Process A holds Resource 1 and wants Resource 2. Process B holds Resource 2 and wants Resource 1. Neither can proceed — a **circular wait** has formed.



Read: Arrow from process to resource = "waiting for". Arrow from resource to process = "assigned to". A cycle = deadlock.

The Four Necessary Conditions (Coffman Conditions)

A deadlock can occur **if and only if** all four conditions hold simultaneously.

1. Mutual Exclusion At least one resource must be held in a non-sharable mode — only one process at a time can use it. If another process requests that resource, it must wait until it is released. (e.g., a printer cannot be shared mid-print.)

2. Hold and Wait A process must be holding at least one resource while simultaneously waiting to acquire additional resources held by other processes. If processes held nothing while waiting, no circular dependency could form.

3. No Preemption Resources cannot be forcibly taken from a process. A resource can only be released voluntarily by the holding process after it completes its task. If the OS could forcibly take resources, it could break any circular wait.

4. Circular Wait There must exist a circular chain of processes $\{P_0, P_1, \dots, P_n\}$ such that P_0 waits for a resource held by P_1 , P_1 waits for one held by P_2 , ..., and P_n waits for one held by P_0 . This closed loop means no process in the cycle can proceed.

Key insight: All four must hold simultaneously. Eliminating even **one** condition prevents deadlock — this is the basis of prevention strategies.

Resource-Allocation Graph (RAG)

The **Resource-Allocation Graph (RAG)** is a formal tool to represent and detect resource allocation state.

It is a directed graph with:

- **Circles** = Processes ($P_1, P_2, \dots$)
- **Rectangles** = Resources ($R_1, R_2, \dots$) with dots inside = instances

Two edge types:

- **Request Edge ($P_i \rightarrow R_j$):** P_i is waiting to acquire an instance of R_j .
- **Assignment Edge ($R_j \rightarrow P_i$):** An instance of R_j is allocated to P_i .

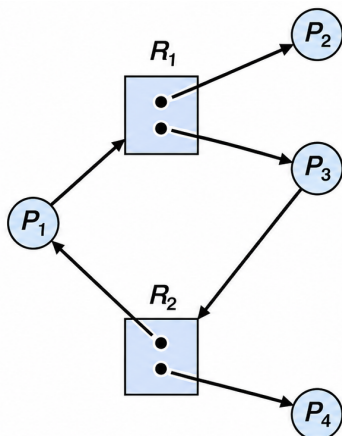


Figure 1: Resource-allocation graph with a cycle but no deadlock.

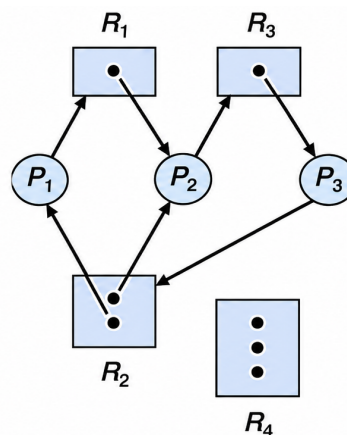


Figure 2: Resource allocation graph with deadlock.

Trace cycles to detect deadlock.

RAG Cycle Rules — Exam Critical:

Situation	Conclusion
No cycle in RAG	No deadlock — guaranteed
Cycle + single instance per resource	Deadlock — certain
Cycle + multiple instances per resource	Deadlock — possible, not certain

Methods for Handling Deadlocks

Strategy	Core Idea	Used In
Prevention	Eliminate one of the 4 conditions permanently	Embedded/RT systems
Avoidance	Dynamically check each request for safe state	Specialized systems
Detection & Recovery	Allow deadlock; detect and break it	Database systems
Ignorance (Ostrich)	Pretend it never happens; reboot if needed	UNIX, Linux, Windows

Deadlock Prevention

Prevention works by designing the system so that **at least one necessary condition can never hold**. Eliminating any single condition makes deadlock impossible.

1. Eliminate Mutual Exclusion

Make all resources sharable. However, this is **not always feasible** — inherently non-sharable resources like printers and tape drives cannot be shared regardless. **Limitation:** Cannot be applied to most practical resources.

2. Eliminate Hold and Wait

Two strategies:

- **Strategy A:** A process must request and receive all resources it needs before starting. If all are not available together, wait without holding anything.
- **Strategy B:** A process may request resources only when it holds none. It must release all held resources before requesting new ones.

Drawbacks: Low resource utilization (resources held idle). Starvation possible — a process needing popular resources may wait indefinitely.

3. Eliminate No Preemption

Allow resources to be forcibly taken. Two methods:

- **Method 1:** If a holding process requests a resource that is unavailable, it must release all currently held resources. It restarts only when it can obtain all required resources at once.
- **Method 2:** If a requested resource is held by another waiting process, preempt it from that process and give it to the requester. If held by a running process, the requester waits. A process restarts only when all its required resources (including preempted ones) are regained.

Applicable to: CPU registers, memory pages — resources whose state can be saved and restored. Not applicable to printers or tape drives.

4. Eliminate Circular Wait (Most Practical)

Assign a **unique numerical priority number** to every resource type. A process may only request resources in **strictly increasing order** of these numbers.

Example: Tape = 1, Disk = 2, Printer = 3. A process holding Disk (2) can only request Printer (3), not Tape (1). Since every process requests in increasing order, the process holding the highest-numbered resource can always complete and release first — circular wait is impossible.

Advantage: Practically implementable, widely used. **Disadvantage:** Requires careful numbering; may reduce utilization if ordering doesn't match usage patterns.

Model Question

An operating system has 3 processes each requiring 2 units of a resource R. What is the minimum number of instances of R required to be present so that the system is deadlock-free? (3 Marks)

Solution

For n processes, each requiring at most m resource instances, the minimum number of instances required to avoid deadlock is:

$$\text{Minimum instances} = n \times (m - 1) + 1$$

Here:

n = 3 processes

m = 2 instances required per process

$$\begin{aligned}\text{Minimum instances} &= 3 \times (2 - 1) + 1 \\ &= 3 \times 1 + 1 \\ &= 4\end{aligned}$$

Therefore, minimum 4 instances of resource R are required to avoid deadlock.

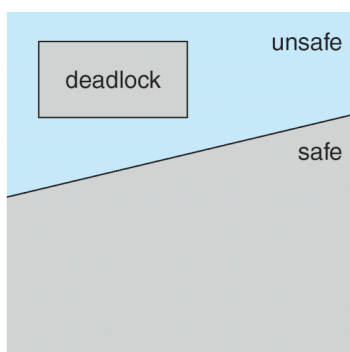
Deadlock Avoidance

Avoidance does not restrict resource usage patterns. Instead, each process **declares its maximum resource needs in advance**. The OS dynamically evaluates each request — if granting it would move the system to an unsafe state, the request is denied even if resources are physically available.

Safe State and Unsafe State

Safe State: A state is safe if there exists a **safe sequence** — an ordering $\langle P_1, P_2, \dots, P_n \rangle$ of all processes such that for each P_i , the resources P_i still needs can be satisfied by: currently available resources + resources held by all P_j that come before P_i . Every process can eventually finish in this order.

Unsafe State: No such safe sequence exists. An unsafe state does **not** guarantee deadlock — it means deadlock is **possible** if processes make unfortunate requests. Avoidance ensures the system never enters an unsafe state.



Safe $\Rightarrow$ No deadlock | Unsafe $\Rightarrow$ Possible deadlock | Deadlock $\subset$ Unsafe

Core principle: Grant a resource request only if the resulting state is still safe. Otherwise, deny and make the process wait.

Safe State Example (12 Tape Drives, 3 Processes)

Process	Max Need	Current Alloc	Remaining Need
P1	10	5	5
P2	4	2	2
P3	9	2	7

$$\text{Available} = 12 - (5+2+2) = 3$$

Safety check at t0:

- Available = 3. P2 needs $2 \leq 3 \rightarrow$ finishes. Releases 2 $\rightarrow$ Available = 5.
- Available = 5. P1 needs $5 \leq 5 \rightarrow$ finishes. Releases 5 $\rightarrow$ Available = 10.
- Available = 10. P3 needs $7 \leq 10 \rightarrow$ finishes.

Safe sequence: $\langle P2, P1, P3 \rangle$ — System is in safe state.

If P3 requests 1 more at t1: Available = 2. After P2: Available = 4. P1 needs 5, P3 needs 6 — neither can finish. **No safe sequence $\rightarrow$ Request denied.**

Deadlock Avoidance Algorithms

Algorithm	Used For	Key Mechanism
RAG Algorithm	Single instance per resource type	Checks if assigning a request edge creates a cycle (uses claim edge)
Banker's Algorithm	Multiple instances per resource type	Safety check using Work/Finish vectors

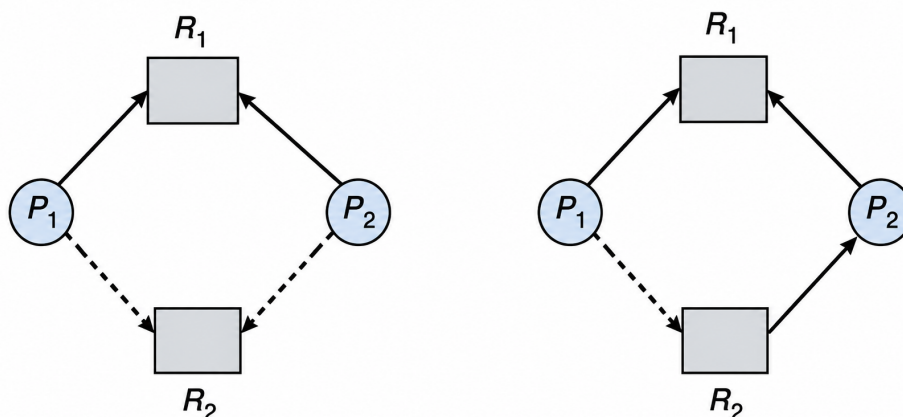
1. Resource Allocation Graph Algorithm

Used only for **single-instance resource types**. Introduces a third edge type: **Claim Edge** ($P_i \text{ ----} \rightarrow R_j$): Drawn as a dashed arrow — indicates P_i **may** request R_j in the future.

Working:

- When P_i actually requests R_j , claim edge converts to a request edge.
- Grant the request **only if** converting the request edge to an assignment edge does **not** create a cycle.
- A cycle means unsafe state $\rightarrow$ deny request.

Example: P2 requests R2. Although R2 is free, granting it would create cycle $P1 \rightarrow R1 \rightarrow P2 \rightarrow R2 \rightarrow P1$ (if P1 has claim on R2 and holds R1). Since this leads to unsafe state, request is **denied**.



2. Banker's Algorithm

Named after banking — a bank never lends money in a way that it cannot satisfy all customers' remaining needs. The OS (bank) never allocates resources (money) such that it cannot satisfy the worst-case demands of all processes (customers).

Applicability: Multiple instances per resource type. Each process declares maximum needs before execution.

Data Structures (n = processes, m = resource types)

Structure	Size	Meaning
Available	1D, length m	$Available[j] = k \rightarrow k$ instances of R_j are currently free
Max	2D, $n \times m$	$Max[i][j] = k \rightarrow P_i$ may request at most k instances of R_j
Allocation	2D, $n \times m$	$Allocation[i][j] = k \rightarrow P_i$ currently holds k instances of R_j
Need	2D, $n \times m$	$Need[i][j] = Max[i][j] - Allocation[i][j] \rightarrow P_i$'s remaining requirement

Key formula: $Need[i][j] = Max[i][j] - Allocation[i][j]$

Safety Algorithm

Purpose: Determine whether the current state is safe.

1. **Initialize:** $Work = Available$. $Finish[i] = false$ for all i .
2. **Find eligible process:** Find i where: (a) $Finish[i] == false$ AND (b) $Need[i] \leq Work$. If no such i , go to step 4.
3. **Simulate completion:** $Work = Work + Allocation[i]$. $Finish[i] = true$. Go to step 2.
4. **Conclude:** If $Finish[i] == true$ for all $i \rightarrow$ Safe state. Otherwise $\rightarrow$ Unsafe state.

Resource-Request Algorithm

Purpose: Decide if a specific request from P_i can be granted.

Let $Request_i$ = request vector of P_i .

1. If $Request_i \leq Need[i] \rightarrow$ continue. Else $\rightarrow$ error (process exceeded max claim).
2. If $Request_i \leq Available \rightarrow$ continue. Else $\rightarrow P_i$ must wait (resources not available).
3. **Trial allocation:**
 - $Available = Available - Request_i$
 - $Allocation[i] = Allocation[i] + Request_i$
 - $Need[i] = Need[i] - Request_i$
4. Run Safety Algorithm on new state. If **safe** $\rightarrow$ grant request. If **unsafe** $\rightarrow$ revert simulation, P_i waits.

Question:

Consider a system with 5 processes P_0 – P_4 and 3 resource types $A=10, B=5, C=7$. The Allocation and Max matrices are given. Is the system in a safe state? If yes, find the safe sequence.

Process	Alloc (A,B,C)	Max (A,B,C)
P_0	0,1,0	7,5,3
P_1	2,0,0	3,2,2
P_2	3,0,2	9,0,2
P_3	2,1,1	2,2,2
P_4	0,0,2	4,3,3

Solution:

Step 1 — Compute Need = Max – Allocation

Process	Need (A,B,C)
P0	7,4,3
P1	1,2,2
P2	6,0,0
P3	0,1,1
P4	4,3,1

Step 2 — Compute Available

Total allocated = (0+2+3+2+0), (1+0+0+1+0), (0+0+2+1+2) = A:7, B:2, C:5

Available = Total – Allocated = (10–7), (5–2), (7–5) = **A:3, B:3, C:2**

Step 3 — Run Safety Algorithm

Initialize: Work = [3,3,2]. Finish = [F,F,F,F,F]

Step	Process Chosen	Condition (Need ≤ Work?)	Work after release	Finish
1	P1	[1,2,2] ≤ [3,3,2] ✓	[3,3,2]+[2,0,0] = [5,3,2]	P1=T
2	P3	[0,1,1] ≤ [5,3,2] ✓	[5,3,2]+[2,1,1] = [7,4,3]	P3=T
3	P4	[4,3,1] ≤ [7,4,3] ✓	[7,4,3]+[0,0,2] = [7,4,5]	P4=T
4	P0	[7,4,3] ≤ [7,4,5] ✓	[7,4,5]+[0,1,0] = [7,5,5]	P0=T
5	P2	[6,0,0] ≤ [7,5,5] ✓	[7,5,5]+[3,0,2] = [10,5,7]	P2=T

All Finish = True.

∴ **System is in SAFE STATE. Safe Sequence: ⟨P1, P3, P4, P0, P2⟩**

Question:

Using the same system as Problem 1, process P1 requests resources (1,0,2). Should this request be granted?

Solution:

Available = [3,3,2], Need[P1] = [1,2,2], Alloc[P1] = [2,0,0]

Step 1 — Check Request ≤ Need[P1] [1,0,2] ≤ [1,2,2] ? → 1≤1 ✓, 0≤2 ✓, 2≤2 ✓ → **Pass**

Step 2 — Check Request ≤ Available [1,0,2] ≤ [3,3,2] ? → 1≤3 ✓, 0≤3 ✓, 2≤2 ✓ → **Pass**

Step 3 — Trial allocation (pretend to grant)

	Before	After
Available	[3,3,2]	[3–1, 3–0, 2–2] = [2,3,0]
Alloc[P1]	[2,0,0]	[2+1, 0+0, 0+2] = [3,0,2]
Need[P1]	[1,2,2]	[1–1, 2–0, 2–2] = [0,2,0]

Step 4 — Run Safety Algorithm on new state

Work = [2,3,0], Finish = [F,F,F,F,F]

Step	Process	Need $\leq$ Work?	Work after
1	P1	$[0,2,0] \leq [2,3,0]$ ✓	$[2,3,0] + [3,0,2] = [5,3,2]$
2	P3	$[0,1,1] \leq [5,3,2]$ ✓	$[5,3,2] + [2,1,1] = [7,4,3]$
3	P4	$[4,3,1] \leq [7,4,3]$ ✓	$[7,4,3] + [0,0,2] = [7,4,5]$
4	P0	$[7,4,3] \leq [7,4,5]$ ✓	$[7,4,5] + [0,1,0] = [7,5,5]$
5	P2	$[6,0,0] \leq [7,5,5]$ ✓	$[10,5,7]$

All Finish = True $\rightarrow$ Safe state exists. $\therefore$ Request (1,0,2) from P1 is GRANTED.

Question:

A system has 4 processes and 3 resources (A=9, B=3, C=6). Check if the system is safe.

Process	Alloc (A,B,C)	Max (A,B,C)
P0	2,1,2	4,2,3
P1	3,0,2	9,1,4
P2	2,1,1	3,2,2
P3	1,1,0	5,3,3

Solution:

Step 1 — Need Matrix

Process	Need (A,B,C)
P0	2,1,1
P1	6,1,2
P2	1,1,1
P3	4,2,3

Step 2 — Available Allocated = (2+3+2+1), (1+0+1+1), (2+2+1+0) = A:8, B:3, C:5 Available = (9-8), (3-3), (6-5) = **A:1, B:0, C:1**

Step 3 — Safety check

Work = [1,0,1]

- P0: Need=[2,1,1] $\leq$ [1,0,1]? $\rightarrow 2 \leq 1$ ✗ $\rightarrow$ Skip
- P1: Need=[6,1,2] $\leq$ [1,0,1]? ✗ $\rightarrow$ Skip
- P2: Need=[1,1,1] $\leq$ [1,0,1]? $\rightarrow 1 \leq 0$ ✗ $\rightarrow$ Skip
- P3: Need=[4,2,3] $\leq$ [1,0,1]? ✗ $\rightarrow$ Skip

No process can be scheduled. Finish = [F,F,F,F]

$\therefore$ System is in UNSAFE STATE — Deadlock is possible.

Question:

A system has 5 processes P0–P4 and 4 resource types (A=10, B=5, C=7, D=8). Is the system safe?

Process	Alloc (A,B,C,D)	Max (A,B,C,D)
P0	2,0,1,1	3,2,1,4
P1	0,1,2,1	0,2,5,2
P2	4,0,0,3	5,1,0,5
P3	0,2,1,0	1,5,3,0
P4	1,0,3,0	3,0,3,3

Solution:**Step 1 — Need Matrix**

Process	Need (A,B,C,D)
P0	1,2,0,3
P1	0,1,3,1
P2	1,1,0,2
P3	1,3,2,0
P4	2,0,0,3

Step 2 — Available Alloc totals = (2+0+4+0+1),(0+1+0+2+0),(1+2+0+1+3),(1+1+3+0+0) = A:7, B:3, C:7, D:5
 Available = (10–7),(5–3),(7–7),(8–5) = **A:3, B:2, C:0, D:3**

Step 3 — Safety check

Work = [3,2,0,3]

Step	Process	Need ≤ Work?	Work after
1	P0	[1,2,0,3] ≤ [3,2,0,3] ✓	[3,2,0,3]+[2,0,1,1]=[5,2,1,4]
2	P2	[1,1,0,2] ≤ [5,2,1,4] ✓	[5,2,1,4]+[4,0,0,3]=[9,2,1,7]
3	P1	[0,1,3,1] ≤ [9,2,1,7]? → 3≤1 ✗	Skip
3	P3	[1,3,2,0] ≤ [9,2,1,7]? → 3≤2 ✗	Skip
3	P4	[2,0,0,3] ≤ [9,2,1,7] ✓	[9,2,1,7]+[1,0,3,0]=[10,2,4,7]
4	P1	[0,1,3,1] ≤ [10,2,4,7] ✓	[10,2,4,7]+[0,1,2,1]=[10,3,6,8]
5	P3	[1,3,2,0] ≤ [10,3,6,8] ✓	All done

All Finish = True.

∴ **System is SAFE. Safe Sequence: ⟨P0, P2, P4, P1, P3⟩**

Advantages of Banker's Algorithm: Prevents deadlock while allowing concurrent execution. Manages multiple resource types.

Disadvantages: Requires advance declaration of max needs. Fixed process count — no dynamic creation. Processes cannot change max declaration mid-execution.

Model Question

A system has four resource types: A (15 instances), B (6 instances), C (9 instances), D (10 instances) The snapshot of the system at time T0 is as follows: Is this system safe?

	<i>Current allocation</i>				<i>Maximum demand</i>			
	A	B	C	D	A	B	C	D
P0	2	0	2	1	9	5	5	5
P1	0	1	1	1	2	2	3	3
P2	4	1	0	2	7	5	4	4
P3	1	0	0	1	3	3	3	2
P4	1	1	0	0	5	2	2	1
P5	1	0	1	1	4	4	4	4

Step 1: Calculate Available Resources

Total Allocated

A: $2 + 0 + 4 + 1 + 1 + 1 = 9$

B: $0 + 1 + 1 + 0 + 1 + 0 = 3$

C: $2 + 1 + 0 + 0 + 0 + 1 = 4$

D: $1 + 1 + 2 + 1 + 0 + 1 = 6$

Available Resources

Available = Total – Allocated

Resource	Available
A	$15 - 9 = 6$
B	$6 - 3 = 3$
C	$9 - 4 = 5$
D	$10 - 6 = 4$

Available Vector = (6, 3, 5, 4)

Step 2: Calculate Need Matrix

Need = Maximum Demand – Allocation

Process	Need (A B C D)
P0	(7, 5, 3, 4)
P1	(2, 1, 2, 2)
P2	(3, 4, 4, 2)
P3	(2, 3, 3, 1)
P4	(4, 1, 2, 1)
P5	(3, 4, 3, 3)

Step 3: Apply Banker's Algorithm

Initial Available = (6, 3, 5, 4)

P1 can execute

Need of P1 = (2, 1, 2, 2) $\leq$ (6, 3, 5, 4)

After execution:

Available = (6, 3, 5, 4) + (0, 1, 1, 1)

New Available = (6, 4, 6, 5)

P2 can execute

Need of P2 = (3, 4, 4, 2) $\leq$ (6, 4, 6, 5)

After execution:

Available = (6, 4, 6, 5) + (4, 1, 0, 2)

New Available = (10, 5, 6, 7)

P0 can execute

Need of P0 = (7, 5, 3, 4) $\leq$ (10, 5, 6, 7)

After execution:

Available = (10, 5, 6, 7) + (2, 0, 2, 1)

New Available = (12, 5, 8, 8)

P3 can execute

Need of P3 = (2, 3, 3, 1) $\leq$ (12, 5, 8, 8)

After execution:

Available = (12, 5, 8, 8) + (1, 0, 0, 1)

New Available = (13, 5, 8, 9)

P4 can execute

Need of P4 = (4, 1, 2, 1) $\leq$ (13, 5, 8, 9)

After execution:

Available = (13, 5, 8, 9) + (1, 1, 0, 0)

New Available = (14, 6, 8, 9)

P5 can execute

Need of P5 = (3, 4, 3, 3) $\leq$ (14, 6, 8, 9)

After execution:

Available = (14, 6, 8, 9) + (1, 0, 1, 1)

New Available = (15, 6, 9, 10)

Safe Sequence: P1 $\rightarrow$ P2 $\rightarrow$ P0 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ P5

Final Answer

Yes, the system is in a safe state. Safe sequence: P1 $\rightarrow$ P2 $\rightarrow$ P0 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ P5

Deadlock Detection

In some systems, neither prevention nor avoidance is used. The OS **allows deadlocks to occur** but periodically runs a detection algorithm. Once detected, corrective action is taken.

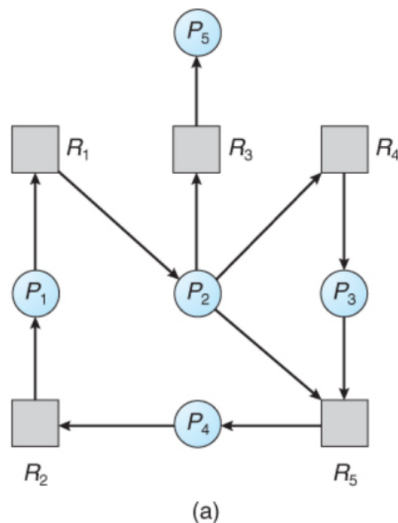
Single Instance — Wait-For Graph

When each resource type has exactly one instance, use the **Wait-For Graph** — derived from the RAG by:

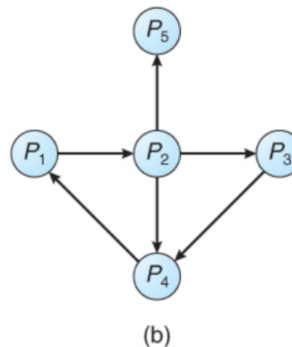
- Removing all resource nodes.
- If P_i was waiting for R_j and R_j was assigned to $P_j \rightarrow$ draw direct edge $P_i \rightarrow P_j$.

Interpretation: $P_i \rightarrow P_j$ means P_i is waiting for a resource currently held by P_j . **Detection rule:** A cycle in the wait-for graph = **deadlock**.

Fig: Wait-For Graph RAG (a):



Wait-For Graph (b):



Multiple Instances — Detection Algorithm

Similar to the Banker's Safety Algorithm, with two key differences:

- Initialize $\text{Finish}[i] = \text{true}$ if $\text{Allocation}[i] = 0$ (process holds nothing $\rightarrow$ can't be deadlocked).
- Uses actual **Request** vector (current real requests) instead of the Need vector.

Steps:

1. $\text{Work} = \text{Available}$. $\text{Finish}[i] = \text{true}$ if $\text{Allocation}[i] = 0$, else false.
2. Find i where: $\text{Finish}[i] == \text{false}$ AND $\text{Request}[i] \leq \text{Work}$. If none, go to step 4.
3. $\text{Work} = \text{Work} + \text{Allocation}[i]$. $\text{Finish}[i] = \text{true}$. Go to step 2.
4. If $\text{Finish}[i] == \text{false}$ for any $i \rightarrow$ **process P_i is deadlocked**. All such processes form the deadlocked set.

When to Run Detection

- **After every failed allocation** — immediate but high overhead.
- **Periodically** — once an hour, or when CPU utilization drops below 40% (sign of many blocked processes).
- **On demand** — when administrator suspects a deadlock.

Question:

A system has processes P_1, P_2, P_3 and resources R_1 (1 instance), R_2 (2 instances), R_3 (1 instance), R_4 (3 instances). The following assignments and requests are made:

- P_1 holds R_2 , requests R_1
- P_2 holds R_1 and R_2 , requests R_3
- P_3 holds R_3

Is there a deadlock?

Solution:

Step 1 — Draw the RAG

P1 —request—► R1 R1 —assigned—► P2
P2 —request—► R3 R2 —assigned—► P1, P2
P3 —holds— R3 R3 —assigned—► P3

Step 2 — Check for cycle

Follow the edges: P1 → R1 → P2 → R3 → P3

P3 holds R3 and is not waiting for anything. P3 can finish and release R3. After P3 releases R3 → R3 becomes free → P2 can get R3 → P2 finishes → releases R1, R2 → P1 can get R1 → P1 finishes.

No cycle exists in an unresolvable sense. ∴ No deadlock. P3 can proceed and unblock the chain.

Question:

Given the following wait situation among processes:

- P1 waits for P2
- P2 waits for P3
- P3 waits for P1
- P4 waits for P2
- P5 waits for P4

Identify all deadlocked processes.

Solution:

Step 1 — Draw Wait-For Graph

P1 → P2 → P3 → P1 (cycle)

P4 → P2

P5 → P4

Step 2 — Detect cycle

Trace: P1 → P2 → P3 → P1 → ... **Cycle found: {P1, P2, P3}**

P4 → P2 (P2 is deadlocked, so P4 waits forever too) P5 → P4 (P4 is blocked, so P5 is blocked too)

∴ Deadlocked processes: {P1, P2, P3, P4, P5} All five processes are deadlocked.

Question:

A system has P1, P2, P3, P4 and resources R1 (2 instances), R2 (2 instances). Given:

- R1 assigned to P1 and P3
- R2 assigned to P2 and P4
- P1 requests R2
- P3 requests R2

Is there a deadlock?

Solution:

Step 1 — RAG

R1 → P1, R1 → P3 (assignments)

R2 → P2, R2 → P4 (assignments)

P1 → R2, P3 → R2 (requests)

Step 2 — Look for cycle P1 → R2 → P2 (no further request from P2) → No cycle through P2 P3 → R2 → P4 (no further request from P4) → No cycle through P4

Step 3 — Check if P2 or P4 can release P2 holds R2 and has no pending request → P2 can finish and release R2. After P2 releases: P1 or P3 can get R2 and finish. P4 holds R2 and has no request → P4 finishes, releases.

∴ **No deadlock — even though R2 is fully allocated, P2 and P4 can release it.** This illustrates: **cycle + multi-instance ≠ always deadlock.**

Recovery from Deadlock

Once a deadlock is detected, the OS must break it. Two approaches:

1. Process Termination

Abort all deadlocked processes: Immediately breaks the deadlock. Drawback — all partial computations are lost and must be recomputed.

Abort one at a time: Abort one process, run detection again, repeat until deadlock is resolved. More careful but incurs repeated detection overhead.

Criteria for selecting which process to abort (minimize cost):

- Priority of the process (lower priority terminated first).
- How long it has run and how close to completion.
- How many and what types of resources it holds.
- How many more resources it needs to complete.
- Number of processes that must be terminated.
- Whether it is interactive or batch.

2. Resource Preemption

Forcibly take resources from one or more deadlocked processes and give them to others until the cycle is broken. Three issues:

1. Selecting a victim: Choose the process from which preemption costs least — consider work done, resources held, ease of restart.

2. Rollback: The victimized process loses its resources and cannot continue. It must be rolled back to a safe checkpoint and restarted. Simplest approach: total rollback (abort and restart from beginning).

3. Starvation: If the same process is always chosen as victim, it may never complete — starvation. Solution: track how many times each process has been victimized and include this count in the cost function. A process should be selected as victim only a finite number of times.

Deadlock Ignorance — Ostrich Algorithm

The approach of simply ignoring the possibility of deadlock. Named the **Ostrich Algorithm** — like an ostrich burying its head in the sand. If a deadlock occurs, the user reboots the system.

This is the **most widely used approach** in practice — employed by **UNIX, Linux, and Windows**. The reasoning: deadlocks are rare, and the overhead of prevention/avoidance is too high for general-purpose systems.

Strategy	Safety	Overhead	Real-World Use
Prevention	High — impossible	Moderate	Embedded/RT systems
Avoidance	High — safe state always	High	Specialized/theoretical
Detection & Recovery	Reactive	Medium	Databases
Ignorance	None	Minimal	UNIX, Linux, Windows

Starvation

A condition where a process is perpetually denied the resources it needs, even though resources are intermittently available, because other processes consistently receive preferential treatment.

Starvation $\neq$ Deadlock:

- In **deadlock**: no process in the set can proceed.
- In **starvation**: one specific process cannot proceed while others continue to run.

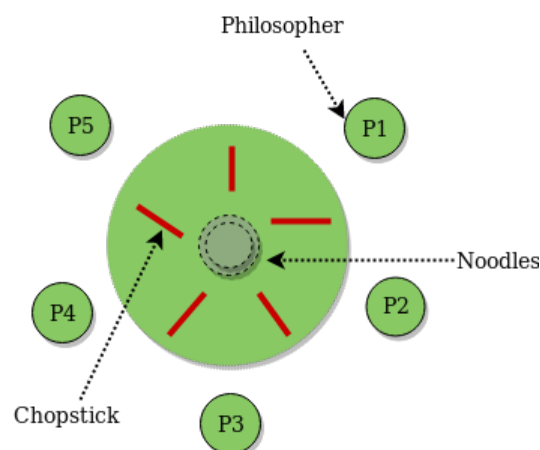
	Deadlock	Starvation
Cause	Circular wait with held resources	Indefinite postponement due to priority
Processes affected	All in the deadlocked set	One or a few low-priority processes
Resolution	Termination or preemption	Aging

Aging: Standard solution — incrementally increase the priority of a process the longer it waits. Eventually, even a low-priority process accumulates enough priority to execute, eliminating starvation.

The Dining Philosophers Problem

Proposed by Dijkstra — a classical problem used to illustrate **deadlock and starvation** in concurrent systems.

Setup: Five philosophers sit around a circular table. One chopstick lies between each pair of adjacent philosophers (5 chopsticks total). A philosopher alternates between **thinking** and **eating**. To eat, they must acquire both the chopstick to their left AND right simultaneously. After eating, they put both down and resume thinking.



Each philosopher needs two adjacent chopsticks to eat.

The problem: If all five simultaneously pick up their left chopstick, none can pick up their right — **deadlock**. If some are always favoured over others — **starvation**.

Naive (Deadlock-Prone) Approach

```
pick up left chopstick
pick up right chopstick
eat
put down both
```

If all five do this simultaneously: all pick up left $\rightarrow$ all wait for right (held by right neighbour) $\rightarrow$ **circular wait** $\rightarrow$ **deadlock**.

Solutions

Solution 1 — Limit to 4 philosophers at a time: Allow at most 4 philosophers to attempt eating simultaneously (enforced by a semaphore initialized to 4). This guarantees at least one philosopher can always acquire both chopsticks, breaking the circular wait.

Solution 2 — Asymmetric Rule (Odd-Even):

- **Odd-numbered philosophers:** pick up left chopstick first, then right.
- **Even-numbered philosophers:** pick up right chopstick first, then left. This breaks symmetry — it is impossible for all to be blocked simultaneously.

Solution 3 — Semaphore with State Checking (Best): A philosopher picks up both chopsticks only when **both are free simultaneously**. Uses a mutex to protect the check-and-pick operation atomically.

States: THINKING, HUNGRY, EATING.

Steps:

1. Enter HUNGRY state.
2. Test: if both left and right neighbours are not EATING, enter EATING state.
3. If not, block on own semaphore.
4. After eating, signal left and right neighbours so they can test.

Guarantees:

- No deadlock — at least one philosopher can always eat.
 - No starvation — every philosopher eventually eats.
 - No two adjacent philosophers eat simultaneously.
-

MODEL QUESTIONS

1. Processes P1 and P2 use the following code for synchronization. Assume flag is a shared variable initialized to false. (3 Marks)

```
if (flag == false)
{
    flag = true;
    criticalSection();
    flag = false;
}
```

Does this guarantee mutual exclusion? Can it lead to deadlock?

2. An operating system has 3 processes each requiring 2 units of a resource R. What is the minimum number of instances of R required to be present so that the system is deadlock-free? (3 Marks)
3. Comment on the following solution to the dining philosophers problem. (3 Marks)
A hungry philosopher first picks up his left fork; if his right fork is also available, he picks up his right fork and starts eating; otherwise he puts down his left fork again and repeats the cycle.
4. A multithreaded web server maintains a cache of recently requested pages using a hash table with 1024 buckets. A hash function maps each URL to a bucket. Each bucket points either to NULL or to a doubly linked list of cached pages that must be searched sequentially. The cache supports three operations: insert, lookup, and delete. It is proposed to use mutex locks for synchronization. Briefly discuss the pros and cons for each of the following locking strategies.
 - (i) One mutex for the entire cache table
 - (ii) One mutex per bucket
 - (iii) One mutex per bucket and one per node in the linked list. (6 Marks)

5. A system has four resource types: A (15 instances), B (6 instances), C (9 instances), D (10 instances) The snapshot of the system at time T0 is as follows: Is this system safe?

	<i>Current allocation</i>				<i>Maximum demand</i>			
	A	B	C	D	A	B	C	D
P0	2	0	2	1	9	5	5	5
P1	0	1	1	1	2	2	3	3
P2	4	1	0	2	7	5	4	4
P3	1	0	0	1	3	3	3	2
P4	1	1	0	0	5	2	2	1
P5	1	0	1	1	4	4	4	4

6. A shared variable y, initially 10, is accessed by two concurrent processes A and B. Process A reads y from memory, adds 5 to it, and writes the result back to memory. Process B reads y from memory, subtracts 3 from it, and writes the result back to memory. Before reading y, each process performs a P (wait) operation on a counting semaphore T. After writing the updated value back to memory, the process performs a V (signal) operation on T. If the semaphore T is initialized to 1, what is the maximum possible value of y after both processes complete execution? Justify your answer. (3 Marks)